

Lightweight RPC for Kotlin Multiplatform

Master Thesis

Student: Aleksandr Stupnikov

Actual supervisor: Ilmir Usmanov

Supervisor: Anton Podkopaev

Problem Statement and Motivation

Kotlin used for **network-heavy shared codebase** software

- Client-server apps, microservices, distributed networks
 - Business logic — unique
 - Network code — repetitive, **boilerplate**
- Client, server, multiple microservices in one project

Problem: network code is too big

Ktor Boilerplate

- Routing logic
- Serialization and deserialization
- HTTP client calls
- Error handling and status codes

Gives **fine control**, but is unnecessary in **shared codebase**

kRPC and gRPC Boilerplate

```
@Rpc interface PizzaShop {  
    suspend fun orderPizza(pizza: Pizza): Receipt  
}  
  
class PizzaShopImpl : PizzaShop {  
    override suspend fun orderPizza(pizza: Pizza): Receipt { .. }  
}  
  
val pizzaShop = client.withService<PizzaShop>()  
pizzaShop.orderPizza(Pizza("Pepperoni"))
```

Drawbacks:

1. Declare and implement an interface just for remote functions
2. Top-level functions cannot be called remotely
3. Remote calls are invisible at the call site

Background — Context Parameters

- Experimental Kotlin feature
- Implicit function parameters resolved from enclosing scope
- Express environmental requirements in type system

```
context(logger: Logger)
fun greet(name: String) {
    logger.info("Hello, $name")
}

fun main() {
    val logger = Logger()
    context(logger) {           // or with(logger)
        greet("World")         // logger is passed implicitly
    }
}
```

Goals and Objectives

Goal: Develop RPC framework for Kotlin Multiplatform shared-codebase projects that reduces boilerplate compared to existing approaches

Objectives:

1. Design RPC framework using **context parameters**
2. Implement as Kotlin compiler plugin with runtime library
3. Evaluate by comparing with alternatives on example projects

Context Parameters for RPC

Context parameters to carry **remote execution configuration** implicitly:

```
@Remote  
context(ctx: RemoteContext<RemoteConfig>)  
suspend fun multiply(lhs: Long, rhs: Long): Long
```

- Determines where the function executes (locally or remotely)
- Carries information to make remote call
- Visible at the type level

RemoteContext Implementation

```
sealed interface RemoteContext<out T : RemoteConfig>
object LocalContext : RemoteContext<Nothing>
class ConfiguredContext<T : RemoteConfig>(val config: T) : RemoteContext<T>
```

- Remote functions can be called in `LocalContext` or `ConfiguredContext<T>`
- `LocalContext` — original body executes locally
- `ConfiguredContext<T: RemoteConfig>` — network query using `T`

How It Looks to the User

```
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = lhs * rhs
```

Client call:

```
fun main() = runBlocking {
    context(ServerConfig.asContext()) {
        println(multiply(6, 7)) // remote call over HTTP
    }
}
```

Server call (on the server):

```
context(LocalContext) { multiply(6, 7) }
```

What Compiler Plugin Generates

User writes:

```
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = lhs * rhs
```

Plugin generates:

```
context(ctx: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) =
    if (ctx is LocalContext) {
        lhs * rhs // original body
    } else {
        (ctx as ConfiguredContext<RemoteConfig>).config.client.call<Long>(
            RemoteCall("multiply", arrayOf(lhs, rhs))
        )
    }
```

Ktor Integration

Framework is **transport-agnostic**, but Ktor integration is provided out of the box

```
embeddedServer(Netty, port = 8080) {  
    install(KRemote) { callableMap = genCallableMap() }  
    routing {  
        remote("/call")  
    }  
}.start()
```

Authentication, logging, CORS, rate limiting — all standard Ktor features work

Example Applications

Written **twice** — once with **Kotlin Remote**, once with **Kotlin RPC**:

- **Todo** (4 operations) — CRUD app
- **Social platform** (36 operations) — 10 microservices backend
- **CMS** (14 operations) — content management system with Ktor authentication

Framework-Specific Code per Application

Application	Kotlin Remote	Kotlin RPC	Boilerplate reduction
Todo (4 ops)	12	12	0%
CMS (14 remote ops)	44	55	20%
Social platform (36 remote ops)	89	136	35%

- Lines counted by IR traversal
 - @Rpc interfaces, implementations, stubs creation
 - context parameters, @Remote annotations, config definitions, context switches
- Savings appear at scale

Boilerplate example

Kotlin Remote

```
@Remote context(_: RemoteContext<UsersService>)
suspend fun getUser(id: Long): User = dep<UserRepository>().get(id)

@Remote context(_: RemoteContext<PostsService>)
suspend fun postWithAuthor(postId: Long) = UsersService.runWith {
    val post = dep<PostRepository>().get(postId)
    PostWithAuthor(post, getUser(post.authorId))
}
```

Kotlin RPC

```
@Rpc interface UsersService {
    suspend fun getUser(id: Long): User
}

@Rpc interface PostsService {
    suspend fun postWithAuthor(postId: Long): PostWithAuthor
}

class UsersServiceImpl : UsersService {
    override suspend fun getUser(id: Long) = userRepo.get(id)
}

class PostsServiceImpl(private val users: UsersService) : PostsService {
    override suspend fun postWithAuthor(postId: Long): PostWithAuthor {
        val post = dep<PostRepository>().get(postId)
        return PostWithAuthor(post, users.getUser(post.authorId))
    }
}
```

Future work

- Code slicing — use `RemoteConfig` types to compile per-node artifacts containing only reachable code
- Bidirectional Streaming
- Remote lambdas

Summary

1. Developed **Kotlin Remote** — RPC framework that uses context parameters for remote calls
2. No interface required, remote calls visible at call site, remote objects support
3. Boilerplate reduction vs Kotlin RPC:
 - Social platform (36 ops): **35% less** framework code
 - CMS (14 ops): **20% less** framework code
 - Todo (4 ops): **tied**
4. No framework-level performance overhead

Limitations

- Shared-codebase Kotlin only — not for independently developed projects
- Depends on experimental context parameters

Performance Comparison

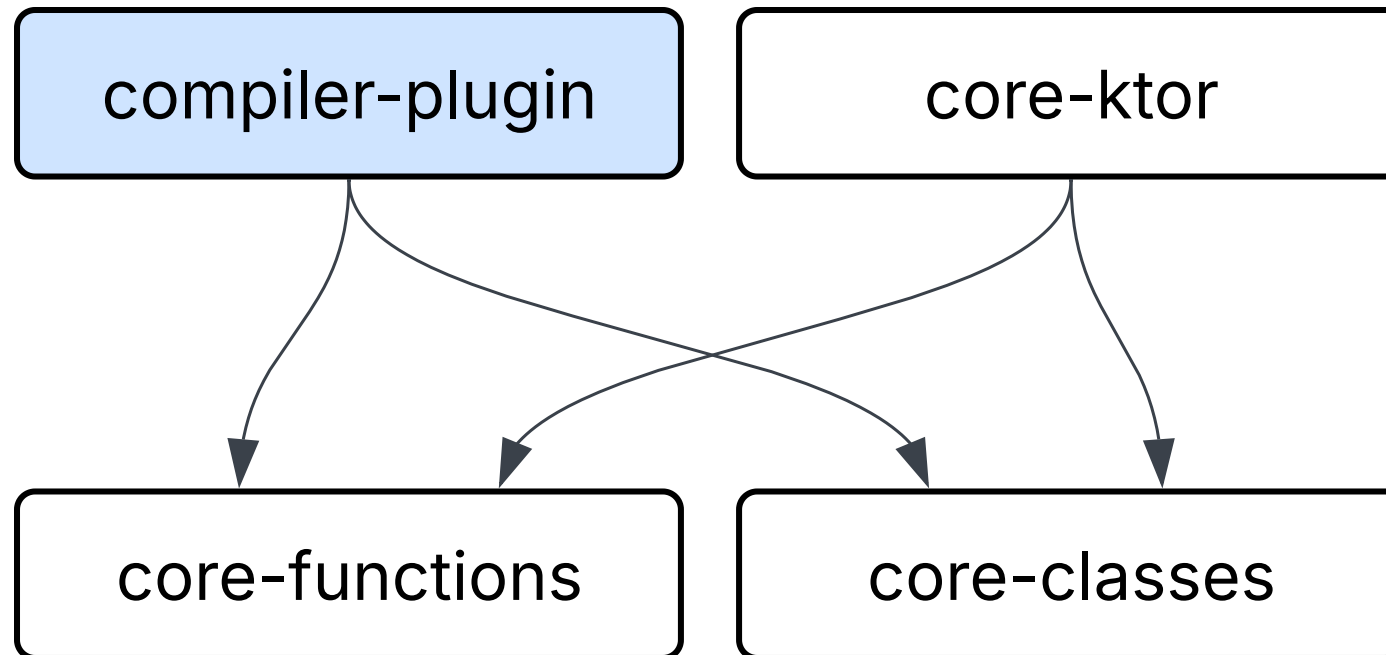
Benchmark	Kotlin Remote	Kotlin RPC
<code>echoLong</code> p50 latency	312 μ s	298 μ s
<code>listTodos(100)</code> p50 latency	393 μ s	339 μ s
Local dispatch overhead	+6.7 ns/call	—
Bytecode per function	~2 190 B	~2 500 B

- Latency gap is transport-level: HTTP/1.1 per-call vs WebSocket — not framework overhead
- Local dispatch in `LocalContext` is one `is`-check, essentially free

Feature Comparison

Feature	gRPC	Kotlin RPC	Kotlin Remote
Service interface required	Yes (IDL)	Yes (<code>@Rpc</code>)	No
Top-level functions	No	No	Yes
Remoteness marked at call site	No	No	Enclosing <code>context</code> block
Stateful remote objects	No	No	Yes
Kotlin Multiplatform support	Limited	Yes	Yes
Streaming	Yes	Yes	No
Cross-language	Yes	No	No

Kotlin Remote module structure



KMP support

No reflection on KMP → need to gather remote functions metadata statically

The plugin replaces `genCallableMap()` intrinsic calls with metadata collected at compile time:

```
// User writes:
val map = genCallableMap()

// Plugin replaces with:
val map = CallableMap(
    "pkg.multiply" to RemoteCallable(
        returnType    = typeOf<Long>(),
        parameters    = arrayOf(typeOf<Long>(), typeOf<Long>()),
        invokator      = { args -> multiply(args[0] as Long, args[1] as Long) }
    ),
    // ... entry for every @Remote function in the source
)
```

No reflection needed — works on **all KMP targets** (JVM, JS, Native, Wasm).

RemoteSerializable classes — Distributed Objects

```
@RemoteSerializable
class Calculator(private var state: Int) {
    @Remote context(_: RemoteContext<RemoteConfig>)
    suspend fun multiply(x: Int): Int { state *= x; return state }

    companion object {
        @Remote context(_: RemoteContext<RemoteConfig>)
        suspend operator fun invoke(init: Int) = Calculator(init)
    }
}
```

```
ServerConfig.runWith {
    val calc = Calculator(5)           // created on server, stub returned
    calc.multiply(6)                   // method runs on server, state preserved
    calc.multiply(7)                   // state = 5 * 6 * 7 = 210
}
```

Remote Classes — How It Works

Serialization:

- Real instance stored in `RemoteInstancesPool` on the server
- Client receives a **stub** (just ID + URL)
- Stub is a generated subclass

Invocation:

- Method call on stub triggers remote call
- Server looks up real instance by ID
- Executes method on the real object

Garbage Collection:

- Lease-based: client periodically renews leases for held stubs
- When leases expire, server removes instance from pool
- Uses weak references to detect when client drops stubs

Client and Server Setup

Client

```
object ServerConfig : RemoteConfig {
    override val client = HttpClient {
        defaultRequest {
            url("http://localhost:8080")
        }
        install(ContentNegotiation) {
            json()
        }
    }.remoteClient(genCallableMap(), "/call")
}

fun main() = runBlocking {
    context(ServerConfig.asContext()) {
        println(multiply(6, 7)) // remote
    }
}
```

Server

```
fun main() {
    embeddedServer(Netty, port = 8080) {
        install(KRemote) {
            callableMap = genCallableMap()
        }
        routing {
            remote("/call")
        }
    }.start(wait = true)
}
```

`genCallableMap()` — compiler intrinsic,
replaced at compile time with metadata for
all `@Remote` functions

What Functions Can Be Remote?

The framework supports a wide range of function kinds:

- **Top-level functions** — `suspend fun multiply(a: Long, b: Long)`
- **Extension functions** — `suspend fun Long.times(rhs: Long)`
- **Class methods** — instance and companion object
- **Nested / local functions** — declared inside other functions
- **Generic functions** — with serializable upper bounds
- **Recursive functions** — recursive calls execute locally
- **Functions that throw** — exceptions propagated transparently

Exception Handling

Remote exceptions are **transparently propagated** to the client:

```
@Remote context(_: RemoteContext<RemoteConfig>)
suspend fun div(x: Int, y: Int) = x / y

fun main() = runBlocking {
    try {
        ServerConfig.runWith { div(10, 0) }
    } catch (e: ArithmeticException) {
        e.printStackTrace()
    }
}
```

```
java.lang.ArithmeticException: / by zero
    at ExceptionKt.div(Exception.kt:11)
    at == Remote Call Boundary ==.(Unknown Source)    ← synthetic frame
    at ExceptionKt.main(Exception.kt:15)
```

Context Hierarchy — Server Capabilities

Contexts form a **subtype hierarchy** expressing server capabilities:

```
interface ArithmeticConfig : RemoteConfig          // basic math
interface TrigonometricConfig : ArithmeticConfig // extends with trigonometry

@Remote context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.mul(rhs: Double) = this * rhs

@Remote context(_: RemoteContext<TrigonometricConfig>)
suspend fun sin(x: Double): Double {
    // can call mul() – ArithmeticConfig is a supertype
    // when on the server: mul() runs locally (LocalContext)
    ...
}
```

Type system enforces which functions are callable from where.

Compiler Plugin — Technical Details

FIR (Frontend)

- Checker: `@Remote` must have `RemoteContext` context param and be `suspend`
- Generator: stub subclass for `@RemoteSerializable`
- Errors visible in IntelliJ IDEA

IR (Backend)

- `RemoteFunctionBodyTransformer` — rewrites bodies
- `CallableMapGenerator` — replaces `genCallableMap()` calls
- `RemoteClassListGenerator` — replaces `genRemoteClassList()` calls

Background — Coeffects

Coeffects — a type-system mechanism for tracking how computations **depend on their environment**.

- Dual of **effects**: effects describe what a computation **does**, coeffects describe what it **requires**
- Implemented via **indexed comonads** in theory
- Practically expressible as **implicit parameters**

| A remote function **requires** a way to reach a remote machine — this is a **coeffect**.

Background — Functional RPC (Servant in Haskell)

```
position :: Int -> Int -> ClientM Position      -- remote procedure
hello    :: Maybe String -> ClientM HelloMessage -- remote procedure

queries :: ClientM (Position, HelloMessage)
queries = do
    pos      <- position 10 10
    message <- hello (Just "servant")
    return (pos, message)

run :: IO ()
run = do
    Right (pos, message) <- runClientM queries ... "localhost"
    print pos
```